

Your Starter Pack to Agentic Automation [Tamil Edition]

Autonomous Agent — Use Case & Problem Statement

AI IT Helpdesk Agent | IT Support & Ticketing Domain

Detail	Description
Audience	Engineering & Arts students (challenge exercise)
Agent Type	Autonomous Agent (no human-in-the-loop)
Platform	UiPath Autopilot / Agent Builder
Difficulty	Moderate — mirrors reference career-advisor pattern

1. Problem Statement

The IT support team at Rajalakshmi Engineering College receives a steady stream of employee and student emails requesting password resets, software access requests, and hardware fault reports. Currently, the helpdesk team manually reads every ticket email, classifies its type, extracts the requester and asset details, and decides whether to auto-resolve, escalate, or log the issue. This manual triage causes delays during peak periods (start of semester, exam server load) and results in inconsistent SLA (Service Level Agreement) tracking.

Design and build an Autonomous UiPath Agent, the AI IT Helpdesk Agent, that reads an incoming IT support email, reasons over its content, classifies it, extracts structured ticket details, and independently completes the required action end-to-end — without any human approval step — by invoking RPA workflows and Generative AI activities as tools.

2. System Prompt

You are an AI IT Helpdesk Agent for Rajalakshmi Engineering College.
Your goal is to analyze IT support emails and decide the best possible action.

Follow this reasoning process:

1. Read and understand the email content.
2. Classify it into one of the following:
 - Password Reset Request
 - Software Access Request
 - Hardware Fault Report
 - Other
3. If it is Password Reset, Software Access, or Hardware Fault:
 - Extract requester name, employee/student ID, and asset or system name
 - Decide if it is urgent (e.g., blocking work/exam access right now)
 - If urgent, recommend immediate ticket creation and notification
 - Generate a professional acknowledgement reply with an estimated SLA
4. If it is Other:
 - Skip processing

Rules:

- Think step-by-step before answering
- Be concise and structured
- Do not assume missing details

- Return output in structured format

3. User Prompt

Analyze the following IT support email and decide the appropriate action:
{{support_email_text}}

4. Input Schema

```
{
  "support_email_text": "string"
}
```

5. Output Schema

```
{
  "category": "string",
  "requester_name": "string",
  "requester_id": "string",
  "asset_or_system": "string",
  "urgency": "string",
  "action": "string"
}
```

6. Tool: Build Workflow & Publish

Create the following process inputs for the RPA workflow the agent will invoke:

- category (text)
- requester_name (text)
- requester_id (text)
- asset_or_system (text)
- urgency (text)

7. Add Tool: Map Inputs

Agent Field	Process Argument
category	in_category
requester_name	in_requester_name
requester_id	in_requester_id
asset_or_system	in_asset_or_system
urgency	in_urgency

8. Updated System Prompt — Tool Invocation Logic

```
If the email is classified as Password Reset Request, Software Access
Request, or Hardware Fault Report:
- Use the create_ticket tool to log the extracted requester and asset
  details in the IT Service Desk system.
- Use the generate_reply GenAI activity to draft a professional
  acknowledgement email to the requester with an estimated SLA.
- If urgency = "High", use the notify_it_team RPA workflow to alert the
  on-call IT support engineer automatically (fully autonomous, no
  manual approval required).

If the email is classified as Other:
- Take no further action.
```

9. Tools & Activities Summary

Tool / Activity	Type
create_ticket	RPA Workflow (Service Desk / Ticketing System Automation)
generate_reply	GenAI Activity (LLM prompt-based text generation)
notify_it_team	RPA Workflow (Outlook / Teams notification)
classify_ticket	GenAI Activity (embedded in Agent reasoning)

Note: This use case is intentionally fully autonomous — the agent completes classification, extraction, ticket logging, notification, and reply generation without an escalation or human-review step, demonstrating end-to-end autonomous decision-making for the workshop challenge.

10. Expected Learning Outcome

- Design a multi-step reasoning system prompt for an autonomous agent
- Define structured input/output schemas for agent-to-workflow communication
- Map agent-extracted fields to RPA process arguments
- Combine RPA workflows and GenAI activities as callable tools within one agent
- Publish and test a complete autonomous agent in UiPath Agent Builder